

Express.js

Complete Interview Revision Guide

Prepared for **Satish Kumar**

SDE-3 Node.js — WeKan Enterprise Solutions

August 2026

*Middleware, routers, the request lifecycle and the production stack —
with the Spring Boot mapping throughout, and how it all leads into NestJS.*

Contents

1. **What Express actually is** — and what raw Node looks like without it
2. **Middleware** — the checkpoint chain, `next()`, and the three levels
3. **Error middleware** — the four-argument rule
4. **Routers** — path joining, params, layered structure
5. **The full request lifecycle** — traced three ways
6. **The production middleware stack**
7. **Express in practice** — what the industry actually does
8. **Express to NestJS** — the complete mapping
9. Appendix — one-liners and self test

Box colours

- **Green** — the shop analogy, read first if a concept is not landing.
- **Yellow** — the specific questions you asked, answered in place.
- **Blue** — the exact sentence to say in the interview.
- **Red** — traps and common mistakes.

1. What Express actually is

Raw Node, without Express

```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/orders" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ orders: [] }));
  } else if (req.url === "/orders" && req.method === "POST") {
    // you must collect the body chunk by chunk, yourself
  } else {
    res.writeHead(404);
    res.end("Not found");
  }
});

server.listen(3000);
```

This works. The pain is visible:

- **Routing is a giant if/else.** 50 endpoints means 50 branches.
- **No URL params.** `/orders/123` — you parse that string yourself.
- **No body parsing.** A POST body arrives as stream chunks. Collect, join, `JSON.parse`, handle errors. Manually. Every time.
- **No shared logic.** Auth on 30 routes means copy-pasting it 30 times.

With Express

```
const express = require("express");
const app = express();

app.use(express.json()); // body parsing – one line

app.get("/orders", (req, res) => {
  res.json({ orders: [] });
});

app.post("/orders", (req, res) => {
  console.log(req.body); // already parsed
  res.status(201).json({ ok: true });
});

app.listen(3000);
```

Same functionality. All the plumbing is gone.

What Express is, precisely

A thin layer on top of Node's `http` module. It does not replace it — it wraps it. Express gives you exactly three things:

1. **Routing** — map a method plus path to a function
2. **Middleware** — a chain of functions every request passes through
3. **Helpers** — `res.json()`, `req.params`, `req.body`, `res.status()`

Express is famously **unopinionated**. It does not tell you how to structure folders, connect a database, or validate input. You decide everything.

The Spring Boot mapping

Spring Boot	Express
<code>@RestController</code>	<code>app.get(...)</code> , <code>app.post(...)</code>
<code>@RequestMapping("/orders")</code>	<code>app.use("/orders", router)</code>
<code>@PathVariable</code>	<code>req.params</code>
<code>@RequestParam</code>	<code>req.query</code>
<code>@RequestBody</code>	<code>req.body</code>
Filters / Interceptors	Middleware
<code>@ExceptionHandler</code>	Error middleware

The big difference. Spring Boot is opinionated and structured — DI, config, validation, ORM integration. Express is a blank sheet: routing and middleware, and you assemble the rest.

That is exactly why NestJS exists — it is Express underneath with Spring Boot's structure bolted on top.

SAY THIS

"Express is a minimal routing and middleware layer over Node's http module. It is unopinionated — it handles routing, middleware and request/response helpers, and leaves architecture entirely to you."

2. Middleware

THE SHOP

A request does not go straight to the handler. It passes through a **line of checkpoints** first.

Priya walks in. Before Ramesh serves her, she passes:

- **Security guard** — is she logged in? (auth)
- **Receptionist** — write her name in the register (logging)
- **Form desk** — unwrap her order form (body parsing)
- **Ramesh** — the actual handler

Each checkpoint either **passes her along** or **stops her**.

```
function logger(req, res, next) {
  console.log(req.method, req.url);
  next();           // pass to the next checkpoint
}

app.use(logger);    // applies to every request

app.get("/orders", (req, res) => {
  res.json({ orders: [] }); // the final handler
});
```

Every middleware has the same three arguments: `req`, `res`, `next`.

next() is the whole mechanism

- Call `next()` → move to the next checkpoint
- Send a response instead → the chain **stops**
- Do neither → **the request hangs forever**

```
function auth(req, res, next) {
  if (!req.headers.authorization) {
    return res.status(401).json({ error: "Not logged in" }); // STOPS here
  }
  req.user = decodeToken(req.headers.authorization);
  next();           // continues
}
```

Middleware can attach things to `req` — note `req.user = ...`. Every later checkpoint sees it. That is how auth passes the user down to your handler.

The three levels

THE BUILDING HAS THREE SECURITY CHECKS

1. **Main gate** — everyone entering the building. This is **app-level**.
2. **Floor entrance** — the Orders floor desk. People going to the Users floor never see it. This is **router-level**.
3. **Room door** — one specific room needs an extra badge. This is **route-level**.

```
// server.js – MAIN GATE
app.use(logger);           // every request, always

app.use("/orders", ordersRouter);
app.use("/users", usersRouter);
```

```
// routes/orders.js – FLOOR ENTRANCE
router.use(auth);          // every /orders route only

router.get("/", getAllOrders);
router.post("/", isAdmin, createOrder); // ROOM DOOR – this route only
```

Level	Written where	Applies to
App	<code>app.use(...)</code> in <code>server.js</code>	Every request to the app
Router	<code>router.use(...)</code> in a router file	Only requests into that router
Route	listed on the route line	Only that one route

It is the **same** `use()` **mechanism** throughout. The only thing that changes is where you write it, and that determines how wide the net is.

YOUR QUESTION — WHY DON'T AUTH AND ISADMIN RUN FOR GET /USERS?

Because **the router is only entered if the path matches**.

```
app.use("/orders", ordersRouter);
```

That line is itself a **condition**. Read it as: "If the URL starts with `/orders`, hand the request to ordersRouter. Otherwise skip this line entirely."

For `GET /users`, Express sees the URL does not start with `/orders` and **never opens that file**. And `router.use(auth)` lives *inside* that router — if the router is never entered, nothing inside it runs.

The floor analogy, precisely: Priya going to the Users floor does not walk past the Orders desk and get waved through. **She never goes to that floor.** The desk is irrelevant to her journey.

Request: GET /users

```
line 1: app.use(logger)           -> no path, matches -> RUN
line 2: app.use("/orders", ...)  -> "/users" != "/orders" -> SKIP entirely
line 3: app.use("/users", ...)   -> match -> enter usersRouter
```

The rule underneath: every `use()` takes an **optional path**. `app.use(logger)` with no path means `"/`, which matches everything. `app.use("/orders", auth)` matches only URLs under that prefix.

So **"router-level" is not a special category**. It is the same mechanism nested one level down. **The nesting is what limits the scope.**

Why you would choose one level over another

- **Logging** → app-level. You want it on everything.
- **Auth** → router-level. `/orders` needs it; `/health` and `/login` must not.
- **Admin check** → route-level. Only delete and create need it.

Put auth at app-level and you have locked yourself out of your own login endpoint. That is the practical reason the levels exist.

Order matters

```
app.get("/orders", handler); // runs before auth is even registered
app.use(auth);               // too late — this route is unprotected
```

Express runs middleware **top to bottom, in registration order**. This is the most common Express bug.

SAY THIS

"Middleware is a chain of functions every request passes through, each receiving req, res and next. Calling next() continues the chain; sending a response ends it. Order of registration is order of execution."

3. Error middleware

The one rule: four arguments

```
app.use((err, req, res, next) => {      // 4 args = error middleware
  res.status(500).json({ error: err.message });
});
```

Express detects error middleware purely by counting arguments. Three args means normal middleware. Four args means error middleware. There is no flag and no decorator — just the count (`fn.length`).

Drop the unused `next` and it silently becomes normal middleware, catching nothing.

```
app.use((err, req, res) => { ... });    // only 3 args – broken
```

How errors reach it

```
app.get("/orders", (req, res) => {
  throw new Error("boom");           // 1. sync throw – auto-caught
});

app.get("/orders", (req, res, next) => {
  next(new Error("boom"));           // 2. pass it to next()
});
```

`next()` with **no argument** means go to the next checkpoint.

`next(err)` with **an argument** means **skip all remaining checkpoints and jump straight to error middleware**.

It must be registered last

```
app.use(express.json());
app.get("/orders", handler);
app.post("/orders", handler);

app.use(errorHandler);    // last, after all routes
```

The async trap

```
app.get("/orders", async (req, res) => {
  const orders = await db.orders.find().toArray();  // throws
  res.json(orders);
});
```

Express 4 will not catch this. Async functions return a rejected promise instead of throwing, and Express 4 ignores the return value. The request hangs until timeout.

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get("/orders", asyncHandler(async (req, res) => { ... }));
```

YOUR QUESTION — WHAT DOES EXPRESS 5 DO INTERNALLY?

Express 4's router, simplified:

```
try {
  handler(req, res, next);    // return value ignored
} catch (err) {
  next(err);                  // only catches sync throws
}
```

Express 5's router:

```
const result = handler(req, res, next);

if (result && typeof result.then === "function") {
  result.catch(next);        // the new line
}
```

That is the whole fix. Express 5 checks whether the handler returned a promise and attaches `.catch(next)` itself — exactly what your wrapper did manually.

And if it is not a promise? The `if` is skipped and nothing changes. A normal non-async handler returns `undefined`, so Express falls back on its outer try/catch, which already handles synchronous throws. **Both cases covered** — Express 5 simply added the missing half.

A realistic error handler

```
app.use((err, req, res, next) => {
  logger.error({ err, url: req.url });

  const status = err.status || 500;
  res.status(status).json({
    error: status === 500 ? "Internal server error" : err.message
  });
});
```

Note the last line — do not leak internal error messages to users on a 500. Log the real error, return a generic one.

SAY THIS

"Express identifies error middleware by its four-argument signature, and it must be registered after all routes. Errors arrive either from a synchronous throw or from next(err) — and in Express 4, async rejections reach neither, which is why async handlers need a .catch(next) wrapper."

4. Routers

The problem: 50 endpoints in one `server.js`. Unmaintainable.

```
// routes/orders.js
const express = require("express");
const router = express.Router();

router.get("/",    getAllOrders);    // becomes GET  /orders
router.get("/:id", getOneOrder);     // becomes GET  /orders/123
router.post("/",    createOrder);     // becomes POST  /orders

module.exports = router;
```

```
// server.js
app.use("/orders", require("./routes/orders"));
app.use("/users",  require("./routes/users"));
```

A Router is a mountable mini-application. It has its own routes and its own middleware, and you mount it at a path prefix.

YOUR QUESTION — WHERE DID /ORDERS GO IN THE ROUTER FILE?

The prefix is set **once, at mount time**. Inside the router, paths are **relative to that prefix**, and Express joins the two.

Mount prefix	Path in router	Final URL
<code>/orders</code>	<code>/</code>	<code>/orders</code>
<code>/orders</code>	<code>/:id</code>	<code>/orders/123</code>
<code>/orders</code>	<code>/:id/items</code>	<code>/orders/123/items</code>
<code>/orders</code>	<code>/pending</code>	<code>/orders/pending</code>

So `"/` inside the router means **the router's own root**, which is `/orders`.

Why it is built this way: the router does not know where it is mounted. That is the point.

```
app.use("/orders",    ordersRouter);
app.use("/api/v2/orders", ordersRouter);  // same router, second prefix
```

The router file never changes. Hardcode `/orders` inside it and this breaks.

The common mistake:

```
router.get("/orders", getAllOrders);  // becomes /orders/orders
```

People write the prefix twice. If a route 404s and you cannot see why, this is usually it.

Route params and query strings

```
router.get("/:id", (req, res) => {
  req.params.id;    // "123" from /orders/123 – always a string
});

// GET /orders?city=Jamshedpur&page=2
req.query.city;    // "Jamshedpur"
req.query.page;    // "2" – also a string, cast it yourself
```

Everything from the URL is a string. No automatic type conversion, unlike Spring's `@PathVariable Long id`. If you need a number, convert and validate it.

Chaining middleware on a route

```
router.get("/:id", auth, orderController.getOne);
//                (1)      (2)
```

Left to right, in order. `auth` runs first, and only if it calls `next()` does the controller run. Chain as many as you like:

```
router.post("/", auth, validateOrder, rateLimit, orderController.create);
```

YOUR QUESTION — CAN I PASS 4 FUNCTIONS TO ROUTER.GET? IS THAT THE "4 ARGS" RULE?

Yes you can, and no it is not related. Two different things are being counted.

What is counted	Meaning
How many functions you pass to a route	Just a chain — run in order
How many parameters one function declares	3 = normal, 4 = error handler

```
router.get("/", a, b, c, d);
//           4 functions, each declaring (req, res, next)
```

Each of those still has **3 parameters**. Express checks `fn.length` — the parameter count of each individual function — not how many you passed.

You can put an error handler in a route chain:

```
router.get("/", auth, handler, (err, req, res, next) => {
  res.status(500).json({ error: "orders failed" });
});
```

If auth or handler errors, this route-specific handler catches it instead of the global one. Occasionally useful, rarely needed.

The layered structure people actually use

```
routes/orders.js      -> path + middleware only
controllers/orders.js -> reads req, calls service, sends res
services/orders.js    -> business logic, no req/res
models/order.js       -> database schema
```

Express will not impose this — you build it. **Why it matters in an interview:** it is the same layering as Spring's Controller → Service → Repository. Saying you keep `req / res` out of the service layer signals you have built something maintainable, not a 2000-line route file.

SAY THIS

"A Router is a mountable mini-application with its own routes and middleware. I mount one per resource at a path prefix, keep routes thin, and push logic into a service layer so business code does not touch req and res."

5. The full request lifecycle

```
// server.js
app.use(express.json());           // 1
app.use(logger);                   // 2
app.use("/orders", ordersRouter);  // 3
app.use("/users", usersRouter);    // 4
app.use(notFoundHandler);          // 5
app.use(errorHandler);             // 6 (4 args)

// routes/orders.js
router.use(auth);                  // A
router.post("/", isAdmin, createOrder); // B, C
```

Trace 1 — POST /orders, success

```
1. express.json()  -> parses body into req.body,   next()
2. logger          -> logs the method and URL,     next()
3. path matches /orders -> ENTER ordersRouter
   A. auth         -> sets req.user,               next()
   B. isAdmin      -> checks req.user.role,        next()
   C. createOrder  -> res.status(201).json(...)    <- RESPONSE SENT
```

The chain ends there. Lines 4, 5 and 6 never run — once a response is sent, Express stops.

Trace 2 — GET /nothing

```
1. express.json()  -> next()
2. logger          -> next()
3. /orders? no     -> skip
4. /users? no      -> skip
5. notFoundHandler -> res.status(404).json(...)
```

Nothing matched, so it falls through to the 404 handler. That is why it sits after all routes.

Trace 3 — POST /orders where createOrder throws

```
1. express.json()  -> next()
2. logger          -> next()
3. enter router
   A. auth         -> next()
   B. isAdmin      -> next()
   C. createOrder  -> throws!
6. errorHandler    -> res.status(500).json(...)
```

Express jumps straight to the four-argument middleware, skipping everything in between — including `notFoundHandler`.

The three rules that explain all of it

- 1. **Top to bottom, left to right.** Registration order is execution order.
- 2. **A response ends the chain.** Nothing after it runs.
- 3. **An error skips ahead** to the nearest four-argument middleware.

Why server.js is ordered exactly that way

Position	Why
<code>express.json()</code> first	Routes need <code>req.body</code> to already exist
<code>logger</code> early	So it logs every request, even ones that 404
Routers in the middle	The actual work
<code>notFoundHandler</code> after routes	Only reached if nothing matched
<code>errorHandler</code> last	Must catch errors from everything above

Move any one of these and something breaks **silently**. That is the most common Express bug, and knowing the reasoning is what an interviewer is checking.

SAY THIS

"Express processes a request as an ordered chain — app middleware, then a matching router's middleware, then route middleware, then the handler. A response ends the chain, and `next(err)` jumps to the nearest four-argument error middleware. So registration order is the app's control flow: body parsing first, routers next, 404 after routes, error handler last."

6. The production middleware stack

```
app.use(helmet()); // security headers
app.use(cors({ origin: "https://app.houseofrinish.com" }));
app.use(express.json({ limit: "1mb" })); // parse JSON bodies
app.use(morgan("combined")); // request logging
app.use(rateLimit({ windowMs: 60000, max: 100 }));
```

Package	Does
helmet	Sets around 12 security headers — XSS, clickjacking, MIME sniffing
cors	Controls which browser origins may call your API
express.json	Parses JSON request bodies into req.body
morgan	Logs every request
express-rate-limit	Caps requests per IP
compression	gzips responses

Three details worth knowing

- express.json() is built in now.** Before Express 4.16 you needed the body-parser package. Saying "I use body-parser" dates you. Say express.json().
- Always set a limit.** The default is 100kb, but be explicit. Without a cap, someone POSTs a 500 MB JSON body, your process runs out of memory, and JSON.parse on it blocks the event loop.
- CORS is a browser rule, not a security feature.** It stops other websites calling your API through a user's browser. It does **not** stop anyone using curl or Postman. Candidates confuse this constantly.

Writing your own — the request ID pattern

```
function requestId(req, res, next) {
  req.id = crypto.randomUUID();
  res.setHeader("X-Request-Id", req.id);
  next();
}

app.use(requestId);
```

Now every log line can carry req.id, so you can trace one request across services. **Mentioning request ID propagation is a strong senior signal.** In production this is usually wired through AsyncLocalStorage so the ID is available without passing it down manually.

SAY THIS

"A production Express setup is helmet for security headers, cors scoped to known origins, express.json() with an explicit size limit, morgan or pino for structured logging, and rate limiting. I would also add a request-ID middleware so logs are traceable across services."

7. Express in practice

Express 4 is still dominant. Express 5 only went stable recently and adoption is slow, so the `asyncHandler` wrapper or the `express-async-errors` package is still what you will find in most codebases. **Say this** — assuming Express 5 everywhere would sound off.

- **Validation is `zod` or `joi`**, applied as middleware. Nobody validates by hand.
- **The standard folder layout** is `routes/ → controllers/ → services/ → models/`. Not enforced by Express, but conventional enough that deviating raises questions in code review.
- **Logging is pino** more often than winston now — it is faster and outputs structured JSON by default. morgan is still common for HTTP access logs specifically.
- **NestJS is winning at the enterprise end.** New large projects increasingly start with Nest; Express stays for small services and existing systems. **That is exactly why WeKan lists both.**

8. Express to NestJS

Everything above is still there. **NestJS runs on top of Express** — it does not replace it. It adds structure.

The problem NestJS solves

Express gives you a blank sheet. On a six-person team over two years that means six different folder structures, no dependency injection, manual wiring everywhere, and hard-to-test code because everything touches `req` and `res`.

Sound familiar? It is exactly why Spring exists for Java.

The same endpoint, both ways

```
// Express
router.get("/:id", auth, async (req, res, next) => {
  try {
    const order = await orderService.findById(req.params.id);
    res.json(order);
  } catch (err) { next(err); }
});

// NestJS
@Controller("orders")
@UseGuards(AuthGuard)
export class OrdersController {
  constructor(private ordersService: OrdersService) {} // injected

  @Get("/:id")
  findOne(@Param("id") id: string) {
    return this.ordersService.findById(id); // just return it
  }
}
```

Look at what disappeared: no `req`, no `res`, no try/catch, no manual service import. NestJS handles serialization and error catching for you.

The complete mapping

Express	NestJS	Spring Boot
<code>router.get("/:id", ...)</code>	<code>@Get("/:id")</code>	<code>@GetMapping("/{id}")</code>
<code>app.use("/orders", router)</code>	<code>@Controller("orders")</code>	<code>@RequestMapping("/orders")</code>
<code>req.params.id</code>	<code>@Param("id")</code>	<code>@PathVariable</code>
<code>req.body</code>	<code>@Body()</code>	<code>@RequestBody</code>
<code>req.query</code>	<code>@Query()</code>	<code>@RequestParam</code>
Middleware	Guard (auth)	Filter / Interceptor
Middleware	Interceptor (logging, transform)	<code>@Around</code> advice
Manual validation	Pipe	<code>@Valid</code>
Error middleware	Exception filter	<code>@ExceptionHandler</code>
<code>require("./service")</code>	Dependency injection	<code>@Autowired</code>
Folder convention	Module	<code>@Configuration</code>

Read the middleware rows carefully. Express has **one** concept — middleware — doing four jobs. NestJS splits it into four named things with defined positions in the request pipeline. **That is the real difference.**

The three things NestJS actually adds

- Dependency injection.** Declare what you need in the constructor; Nest provides it. Testing becomes trivial — inject a mock service instead of a real one. Biggest win, and identical to Spring.
- Modules.** Code grouped into `OrdersModule`, `UsersModule`, each declaring what it needs and what it exposes. Structure enforced, not conventional.
- TypeScript-first with decorators.** Types on request bodies, automatic validation via pipes, generated OpenAPI docs.

SAY THIS — THE HONEST, STRONG VERSION

"I have built primarily in Express, so I know the middleware pipeline well — and my Spring Boot background maps closely onto NestJS. Nest is Express underneath with a structured layer on top: controllers with decorators instead of route handlers, dependency injection instead of manual imports, and guards, interceptors, pipes and exception filters replacing what Express does with generic middleware. The DI and module model is essentially Spring's, which is why I would be productive quickly."

This is honest and strong. You are not claiming NestJS depth — you are showing you understand the model and can name every piece.

Appendix — One-liners and self test

Every one-liner

Topic	The sentence
What Express is	A minimal routing and middleware layer over Node's http module. Unopinionated — it leaves architecture entirely to you.
Middleware	A chain of functions every request passes through, each receiving req, res and next. next() continues; sending a response ends it. Registration order is execution order.
Levels	App-level runs for everything, router-level only for requests that entered that router, route-level only for that one route. Same use() mechanism, different nesting depth.
Error middleware	Identified by its four-argument signature and registered after all routes. Errors arrive from a sync throw or from next(err).
Express 4 async	Express 4 only catches synchronous throws, so an async rejection escapes and the request hangs. Wrap with .catch(next) or use Express 5.
Routers	A mountable mini-application. The prefix is set once at mount time; paths inside are relative to it.
Lifecycle	App middleware, then matching router middleware, then route middleware, then the handler. A response ends the chain; next(err) jumps to the error handler.
Production stack	helmet, cors scoped to known origins, express.json with an explicit limit, structured logging, rate limiting, and a request-ID middleware.
CORS	A browser rule, not a security feature. It does not stop curl or Postman.
NestJS	Express underneath with structure on top — DI, modules, and guards/interceptors/pipes/filters replacing generic middleware. The DI model is essentially Spring's.

Rapid-fire self test

Question	Where
Name the three things Express gives you over raw http .	1
What are the three parameters of every middleware, and what does each next() outcome do?	2
Request is GET /users . Does router.use(auth) inside the orders router run? Why not?	2
Why is auth usually router-level rather than app-level?	2
How does Express know a function is error middleware?	3
Two ways an error reaches the error handler.	3
Exactly what did Express 5 add to its router?	3
Mount prefix /orders , route path /:id/items . What is the final URL?	4
Is router.get("/", a, b, c, d) valid? Is that the four-argument rule?	4
Why must express.json() come before the routers?	5
Why does notFoundHandler go after the routes but before the error handler?	5
Why set an explicit limit on express.json() ?	6
Does CORS protect your API from Postman?	6
Which four NestJS concepts replace Express middleware?	8